Movie Recommendation Hub: An AI-Powered, Microservices-Based Platform on Self-Hosted Kubernetes

PSIA 2026/2027 – Project Report – **Group 17**

João Russo Tiago Fonseca **[TODO:** Name**]**
ist1**[TODO:** xxxxx**]** ist112346 ist1**[TODO:** xxxxx**]**
Instituto Superior Técnico, Universidade de Lisboa, Lisbon, Portugal

Video: `https://youtu.be/TODO` | **Repository:** `https://gitlab.rnl.tecnico.ulisboa.pt/psia/groups26/g17`

1 INTRODUCTION

The goal of the PSIA project is to deploy a realistic, tiered, microservices-based application with AI components on a public cloud, using Infrastructure-as-Code (IaC) end to end: virtual machines are provisioned with Terraform, configured with Ansible, and orchestrated by a *self-hosted* Kubernetes cluster (managed offerings such as GKE are not allowed). The system must also be instrumented and observed with a monitoring stack, and extended with one advanced feature.

Our application is a *Movie Recommendation Hub*. Users register and log in through Keycloak (OpenID Connect [5]), keep a profile with their favourite genres, watched movies and the users they follow, browse and search a catalogue of about 45 000 movies from The Movies Dataset [1], and view other users' profiles to discover new interests. An AI inference engine, a small open language model (Qwen2.5 3B) served by Ollama on the VMs' CPUs, together with a sentence-embedding model (`nomic-embed-text`) for search by meaning, produces personalised recommendations in two ways: *on demand*, when a user asks for suggestions, optionally with free-text constraints ("a horror movie from the 80s"); and *proactively*, through a background agent that precomputes and caches recommendations per profile, so that users are greeted with fresh suggestions as soon as they log in. Following the database-per-service pattern [4], each backend service owns its own PostgreSQL database.

The system runs on Google Cloud Platform (region `europe-west4`), on a kubeadm cluster of three VMs (one control-plane node and two workers), and is monitored with Prometheus [3] and Grafana. As advanced feature we chose *persistent state*: every VM of the deployment can be shut down and restarted without losing any application state.

The rest of this report is organised as follows. Section 2 presents the architecture, services, data and deployment pipeline. Section 3 describes the persistent-state design and its validation. Section 4 studies throughput and latency as the number of concurrent clients grows, and Section 5 concludes. The demonstration video is available at `https://youtu.be/TODO`, and the source code and deployment instructions are in the group repository at `https://gitlab.rnl.tecnico.ulisboa.pt/psia/groups26/g17`.

2 SYSTEM DESIGN AND ARCHITECTURE

The system follows the tiered microservices architecture of the specification (Fig. 1). The *frontend tier* is a Next.js web application, the only entry point exposed to users; in the cluster it sits behind an NGINX reverse proxy that terminates TLS and routes `/` to the frontend, `/api/*` to the services, `/auth/` to Keycloak and `/grafana/` to Grafana. Users register and log in through *Keycloak*, an OpenID Connect (OIDC) identity provider: the browser obtains a short-lived (5 min) JWT access token and sends it as a `Bearer` header to every API call. The *backend tier* consists of four Quarkus services: the *Movies API* (catalogue browsing, full-text search and search by meaning), the *Profiles API* (favourite genres, watched and saved movies, follow graph), the *Recommendation API*, which stores recommendations and queues the work, and the *recommendation worker*, which consumes that work from *Apache Kafka* and queries the *inference engine*, Ollama. Each API validates the token's signature, issuer and its own audience (`movies-service`, `profiles-service`, `recommendation-service`), so tokens minted for other clients are rejected; the recommendation service and the worker call the other services with their own Keycloak service-account tokens. The *data tier* is PostgreSQL 16, with one instance per service (movies, profiles, recommendations) and one for Keycloak; each is migrated by Flyway, and services read each other's data only through their APIs. A *monitoring server* (Prometheus, Grafana) scrapes all components. Table 1 summarises the components and their status.

2.1 Backend Services and Data

The four services use Java 21+, Quarkus 3.39 (`quarkus-rest`, Hibernate ORM with Panache, Hibernate Validator, SmallRye Reactive Messaging for Kafka) and PostgreSQL 16. Flyway owns each schema; Hibernate only validates entities against them. Table 2 summarises the REST API.

Data. The catalogue comes from *The Movies Dataset* (TMDB, Kaggle). Only `movies_metadata.csv` is used. It is loaded as text into a staging table and cleaned into `movie`, `genre` and an *m*:*n* `movie_genre` bridge in a `movies` schema. Cleaning drops 3 column-shifted rows and 29 duplicate ids, sets blank descriptions to `NULL` and parses the Python-literal genre lists. The result is 45 430 movies and 20 genres, shipped as a `pg_dump` (`data/movies_dump.sql`).

Movies service. `GET /movies` uses Panache paging (`page`, `size`$\leq$100) and returns `{items, page, size, totalItems, totalPages}`. Sorting puts `NULL`s last and breaks ties by id, so pages never overlap. Migrations V1–V6 add the base schema, two stored generated `tsvector` columns with GIN indexes (V2), a `pg_trgm` trigram index on titles (V3), IMDb ids (V4), accent-

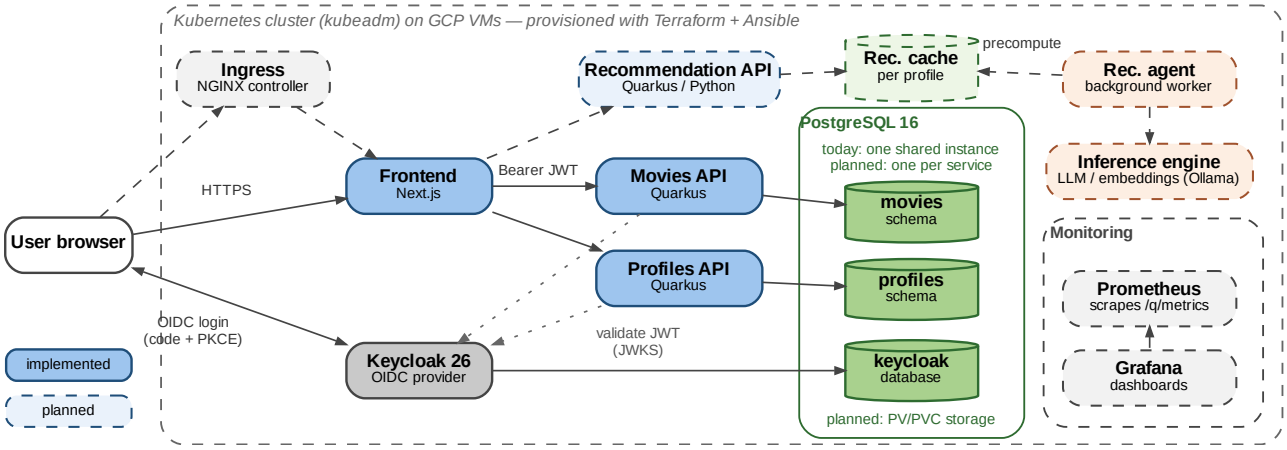


Figure 1: Architecture of the Movie Recommendation Hub. Colours: blue = application services, green = data, orange = AI, grey = infrastructure/operations (dotted arrows are JWT validation). All components shown are now implemented: four Quarkus services, one PostgreSQL instance per service, Kafka between the recommendation service and its worker, Ollama as inference engine, and an NGINX reverse proxy in front of a self-hosted Kubernetes cluster. **[TODO:** regenerate this figure (`figures/architecture_gen.py`, rendered with `neato`): it still draws the planned components with dashed outlines and omits Kafka and the worker.**]**

Table 1: System components and implementation status.

Component	Technology	Status
Frontend	Next.js 16, React 19, Tailwind 4, keycloak-js	Done
Identity provider	Keycloak 26 (OIDC, PKCE)	Done
Movies API	Quarkus 3.39, Hibernate Panache, Flyway, pgvector	Done
Profiles API	Quarkus 3.39, Hibernate Panache, Flyway	Done
Recommendation API + agent	Quarkus, Kafka, background agent	Done
Recommendation worker	Quarkus, Kafka consumer	Done
Inference engine	Ollama (CPU), Qwen2.5 3B, nomic-embed-text	Done
Message queue	Apache Kafka 4.3 (Strimzi, 3 brokers)	Done
Databases	PostgreSQL 16, one instance per service	Done
Container images	Multi-stage Dockerfiles (backend, frontend)	Done
Local deployment	Docker Compose (infra, backend, frontend), `just`	Done
Cluster (IaC)	Terraform, Ansible, kubeadm on GCP	Done
Reverse proxy	NGINX with TLS	Done
Monitoring	Prometheus, Grafana, Micrometer	Done
Persistent state (advanced)	StatefulSets, local-path volumes	Partial

insensitive search via an `IMMUTABLE unaccent` wrapper (V5), and the `pgvector` extension with an embeddings table (V6). In V2, `search_vector` holds English-stemmed title (weight A) and description (weight B). `title_vector` uses the `simple` configuration, so titles made only of stop words (*It*, *Up*) stay findable. A search parses `q` with `websearch_to_tsquery`, which supports phrases, `or` and `-word`. For plain queries, title words are also matched as prefixes (`star wa`→*Star Wars*). If fewer than 10 movies match, trigram `word_similarity` adds similar titles (`godfater`→*The Godfather*). Results are ranked in four tiers: title match, title prefix, description-only match (`ts_rank_cd`), then similar title. Within the first two tiers, movies are ordered by vote count, and an exact title match counts ten times. For Ask AI, the service also searches *by meaning*: a background job embeds the ∼9 100 movies with at least 50 votes (768-dimensional vectors from `nomic-embed-text`, computed by Ollama), and a request is ranked by vector similarity to its embedded text.

Profiles service. No registration hook is needed. On every authenticated request, `CurrentUser` upserts `profiles.profile` (`INSERT ... ON CONFLICT`) from the JWT `sub` (the primary key, a UUID) and `preferred_username`. The schema stores favourite genres, watched movies (with timestamp), a watchlist (V2) and a follow graph, which forbids self-follows through a `CHECK` and is indexed in both directions. Any user can read any profile, but writes go only through `/profiles/me/...`; the watchlist is visible only to its owner. The `PUT`s are idempotent (`ON CONFLICT DO NOTHING`, or full replacement for genres), and `DELETE` undoes them. Every change updates a `last_changed_at` column (V3), and internal endpoints, restricted to a service role, expose profiles and a fingerprint of each one's inputs to the recommendation service.

Persistence per service. Each service owns its own PostgreSQL instance, which no other service connects to (in the cluster, a NetworkPolicy enforces it). There are no foreign keys across databases: the profiles schema stores movie and genre ids only, and the service validates them through the Movies API before storing them. The Recommendation service stores its requests and results in its own database, and the worker has none. Keycloak keeps its data in a separate database.

Recommendation service and inference engine. Making a set of ten recommendations takes the model minutes on a CPU, so the work is asynchronous. The Recommendation API answers at once (202) and puts a job, with a snapshot of the profile, on a Kafka topic (`recommendation.requests`, 6 partitions, keyed by user id). The worker takes jobs one at a time and publishes the result on `recommendation.results`; the API stores it and a third topic (`recommendation.ready`) makes every API instance notify its connected browsers through server-sent events. A job leaves the queue only when it is done,

Table 2: REST endpoints (all require a bearer token).

Endpoint	Purpose
Movies service (:8080)	
`GET /movies`	Paginated list; `sort`: date, id, title, rating
`GET /movies/search?q=`	Ranked full-text + fuzzy search
`GET /movies/{id}`	One movie, or 404
`GET /genres`	The 20 genres
Profiles service (:8081); `{id}` may be `me`	
`GET /profiles?q=`	Username prefix search
`GET /profiles/{id}`	Profile with genres and counters
`PUT .../me/favorite-genres`	Replace favourite genres
`GET .../{id}/watched`	Watched movies, newest first
`PUT|DELETE .../me/watched/{mid}`	Mark/unmark as watched
`PUT|DELETE .../me/watchlist/{mid}`	Save for later / remove
`GET .../{id}/following|followers`	Profiles followed / followers
`PUT|DELETE .../me/following/{id}`	Follow/unfollow
Recommendation service (:8082)	
`GET /recommendations/me`	Recommended for you: `READY`, `PREPARING` or `FAILED`
`POST /recommendations/requests`	Ask AI: queues a request, answers 202
`GET /recommendations/requests[/{id}]`	Last five answers; one request
`GET /recommendations/events`	Server-sent event when a result is stored

so a worker that stops mid-job leaves it to the next one. There are two kinds of recommendations: *Recommended for you*, made from a profile's genres, watched and saved movies and what the people it follows watched, and *Ask AI*, made from free text such as "a horror movie from the 80s" and excluding movies already watched or saved. A 45 000-movie catalogue is too large for a model to read, so the worker narrows it first: it turns an Ask AI request into search filters (genres, years, keywords), the Movies API returns 30 candidates (closest in meaning, then best rated), up to 20 more come from the followees' watched movies, and Qwen2.5 3B, running in Ollama, chooses ten. Its answer must follow a JSON schema that only allows the candidates' ids, so it cannot invent movies; if it chooses fewer than ten, the best-ranked candidates fill the gaps.

A *background agent* (every 10 minutes) keeps Recommended for you current without redoing work. Each result records a fingerprint of the profile it was made from, and the agent queues a new set only for profiles whose inputs changed (once they stop changing for 2 minutes), at most 20 per run; failures are retried with exponential back-off from 30 minutes to 6 hours. Identical inputs share results. The service keeps the current set per user and the last five Ask AI answers in PostgreSQL, and a request still pending after an hour is failed as a safety net for a job lost from the queue. The recommendation service and the worker call the other services as themselves, with Keycloak service-account tokens (client credentials grant) and not on behalf of a user.

Security and operations. Keycloak 26 hosts a `movies` realm, imported from JSON, with self-registration and brute-force protection. Its two *public* clients, `movies-frontend` and `movies-swagger-ui`, use the authorization-code flow with PKCE (S256). Audience mappers add the services' audiences to every token. Each API is an OIDC `service` that checks signature, issuer, expiry and its own audience, so tokens issued for other clients in the realm are rejected. Access tokens live 5 minutes, and the frontend refreshes them. Each service exposes `/q/health` (SmallRye Health), Prometheus metrics at `/q/metrics` (Micrometer) and OpenAPI with a Swagger UI that logs in through Keycloak.

Testing. Tests use Quarkus Dev Services, which start a throwaway PostgreSQL container via Testcontainers. Flyway builds the schema there and loads fixtures from `db/testdata`. Keycloak is replaced by `@TestSecurity`/`@OidcSecurity`, which inject identities and claims. A separate test checks that unauthenticated requests get 401. For the recommendation services, Kafka is replaced by in-memory channels and the other services by mocks; the worker's tests need neither Docker nor Ollama, because the model is a mock.

2.2 Frontend

The web interface is a Next.js 16 (App Router) application written in TypeScript with React 19 and Tailwind CSS 4. All pages are client components that call the APIs directly from the browser. The landing page (`/`) offers *Log in* and *Register*, both handled by Keycloak's hosted pages. The authenticated area comprises `/movies` (catalogue grid with search and sorting, topped by *Recommended for you*), `/movies/[id]` (details, IMDb/TMDB links, *Mark as watched*, *Watch later*), `/recommendations` (*Ask AI*), `/users` (username prefix search) and `/users/[id]` (favourite genres, watched movies, following and followers tabs, *Follow*, and genre editing and the watchlist on one's own profile, `/users/me`).

Authentication uses `keycloak-js` with the public `movies-frontend` client, authorization code flow and PKCE (S256). A route-group layout wraps every authenticated page in an `AuthGate` that initialises Keycloak with `login-required`, redirects anonymous users to the login page and back, and immediately requests `/profiles/me` so that the user's profile is created on first login. The token is kept only in memory; before every request, `lib/api.ts` calls `updateToken(30)`, refreshing it if it expires within 30 s, and maps API errors to typed exceptions.

Lists use a generic `useInfinitePages` hook: an `IntersectionObserver` on a sentinel element fetches the next page (24 movies) ahead of the viewport, guarding against duplicate requests. Search and sort state lives in the URL query (e.g. `?q=batman&sort=newest`), so reloads and the back button preserve it. API and Keycloak locations are `NEXT_PUBLIC_*` variables inlined at build time. Locally the frontend and the APIs are on different origins, so each Quarkus service whitelists the frontend origin via CORS; in the cluster the image is built with relative addresses (`/api`, `/auth`), so the browser sees one origin and the same image works at any address.

Recommendations take minutes, so the interface never waits for them. *Recommended for you* is the first row of

/movies: the first set is requested on the user's first visit, and the row fills in, and updates itself, when the background agent finishes a new set. *Ask AI* sends a free-text request, disables the box while one is being prepared, lists the last five answers, and the user can leave the page: the browser holds a server-sent-events connection to the Recommendation API, and a notification says when a result is ready or why it failed.

2.3 Deployment and Operations

Local deployment. The whole system runs locally in Docker, as three Compose projects that share one network (movies-net): infra/compose.yaml (databases, Kafka, Ollama, Keycloak), backend/compose.yaml (the four Quarkus services) and frontend/compose.yaml (the website). The infrastructure project runs three PostgreSQL 16 containers (the movies one with the pgvector extension), a single-broker Kafka in KRaft mode, Ollama with a one-shot container that downloads the two models, and Keycloak 26 (start-dev -import-realm), which imports the movies realm from movies-realm.json and keeps its state in its own database in the movies server. The movies container mounts infra/init/, whose script creates the keycloak role and database on first start. Every container has a health check (pg_isready, Kafka's broker API, Ollama's list, the /health/ready endpoints). Because depends_on cannot cross projects, a justfile starts them in order and waits for each to report healthy (-wait): just local-first-up also imports the pg_dump catalogue before the backend starts, and just local-up skips the import. In containers the services run in Quarkus' production profile and read their database URLs, OIDC issuer and CORS origins from environment variables; Keycloak keeps the browser-facing address as the token issuer, while the services reach it by container name through its backchannel hostname setting.

Containerization. A single multi-stage backend/Dockerfile builds any of the four Quarkus services from source (Maven in the build stage, a UBI9 OpenJDK 21 runtime in the final one), and frontend/Dockerfile builds the Next.js standalone server. The same Dockerfiles serve Compose and the cloud deployment, where build-images.sh builds the five images for linux/amd64 and pushes them to Docker Hub.

Kubernetes on GCP. The production target is a self-hosted cluster (managed offerings such as GKE are excluded by the specification), created entirely through IaC and driven from a management container that holds Terraform, Ansible, kubectl and gcloud. Terraform creates a VPC and subnet, firewall rules, a reserved static IP and three Compute Engine VMs in europe-west4-b running Ubuntu 24.04: one control-plane node (e2-standard-2) and two workers (e2-standard-4); it also writes Ansible's inventory. Four Ansible playbooks follow: they prepare the nodes (containerd, kubeadm, kubelet), run kubeadm init with Calico 3.32 as pod network and join the workers; install the platform (local-path-provisioner as default storage, the Strimzi Kafka operator and kube-prometheus-stack through Helm); and deploy the application from Kubernetes manifest templates. The application namespace holds one PostgreSQL StatefulSet per service (each with a postgres_exporter sidecar), a three-node Kafka cluster in KRaft mode (topics replicated three times), Ollama with a volume for the models and a Job that downloads them, Keycloak in production mode (realm imported from a ConfigMap), a Deployment per service with startup, readiness and liveness probes on /q/health, and NetworkPolicies that let each database accept connections only from its own service and Ollama only from the worker and the Movies service. NGINX runs on the worker that owns the static IP and is the only public entry point: it terminates TLS (a self-signed certificate unless one is provided) and routes /, /api/*, /auth/ and /grafana/. Secrets are generated on the first run and kept out of git. A full deployment from nothing takes about 25 minutes.

Monitoring. Prometheus and Grafana come from kube-prometheus-stack. Prometheus scrapes the Quarkus services' /q/metrics endpoint (Micrometer: HTTP request rate and latency, JVM and connection-pool metrics), the databases' exporters, Kafka Exporter (consumer lag and topic offsets), NGINX, Keycloak, node-exporter (CPU, memory, disk and network per VM) and kube-state-metrics (pod status, restarts). Because Ollama has no metrics endpoint, the worker measures each model call itself (duration and tokens). A provisioned Grafana dashboard, *Movies*, shows request rates and latency per API, Ollama's answer times and tokens per second, CPU and memory, Kafka's backlog for the worker, the databases' connections and the nodes. Grafana is served at /grafana behind the same address; Prometheus is not public and is reached with a port-forward. **[TODO:** figure of the dashboard under load.]

Reproducibility. Step-by-step instructions (prerequisites, configuration defaults, data loading, troubleshooting) are maintained in the repository README.md and infra/README.md. Locally, just local-first-up starts the whole stack (just local-up on later runs). In the cloud, just images pushes the images, and just all, run inside the management container, runs Terraform and the three playbooks in order; just destroy removes every resource.

3 ADVANCED COMPONENT: PERSISTENT STATE

Goal. All VMs of the deployment (control plane and workers) can be stopped, as when the cloud budget is paused overnight, and started again later. After the cluster converges, every user, profile, follow, watched movie and cached recommendation must still be there, without re-seeding or manual steps. Storage must be self-hosted inside the cluster rather than a managed database service, which also avoids vendor lock-in.

Stateful components. Table 3 lists the state to preserve. Locally, every component keeps its data in its own named Docker volume, which survives docker compose down; in the cluster, each one gets a PersistentVolumeClaim. Keycloak stores the realm and users in PostgreSQL rather than in its embedded H2 store. Everything else (APIs, worker, frontend, NGINX) is stateless and can be rescheduled freely.

Storage options. Each database runs as a single-replica StatefulSet whose volumeClaimTemplate binds a PersistentVolumeClaim (PVC) to a PersistentVolume (PV) [6]. The question is what backs the PVs:

Table 3: Stateful components and how their state persists.

Component (state)	Mechanism	Status
movies DB (catalogue, embeddings)	StatefulSet + PVC, 10 GiB	Configured
profiles DB (genres, watched, follows)	StatefulSet + PVC, 2 GiB	Configured
recommendations DB (results)	StatefulSet + PVC, 2 GiB	Configured
Keycloak DB (realm, users)	StatefulSet + PVC, 2 GiB	Configured
Kafka (queued jobs, offsets)	3 brokers, PVC 5 GiB each, kept on delete	Configured
Inference engine (model weights)	PVC 15 GiB, avoids re-download	Configured
Prometheus TSDB	PVC 10 GiB, 7 days	Configured
Kubernetes (etcd, certificates)	VM boot disk	[TODO:]
Local Docker Compose (all components)	one named volume each	Done

- *Local PVs* on a disk of a given node: simple and fast, but each pod is pinned to that node through node affinity, and the data has no replica.
- *Longhorn* [7]: a CNCF block-storage system that runs as a CSI driver on the cluster nodes, with dynamic provisioning, replicas on several nodes, snapshots and backups. It requires `open-iscsi` on the nodes and some CPU and memory overhead.
- *Rook-Ceph*: more capable, but needs raw disks and far more resources than a few small VMs can spare.
- *NFS server VM*: easy to set up, but a single point of failure, and PostgreSQL over NFS is fragile (locking, `fsync`).

We use local PVs, provided by Rancher's local-path-provisioner, which the platform playbook installs and sets as the default `StorageClass`. It adds no components or memory overhead to small VMs, and the volumes live on the nodes' boot disks, which survive stopping the VMs. The trade-off is that each pod stays pinned to its node and its data has no replica: state survives a restart of the cluster, but not the loss of a VM or a `terraform destroy`. Longhorn would lift that limit at the cost of extra components on every node.

Design. Persistence rests on four measures. (i) *Volumes outlive pods*: each database and each Kafka broker is a StatefulSet with a `volumeClaimTemplate` (Kafka's claims are kept even if the cluster resource is deleted), and the models and Prometheus data have their own claims. (ii) *Initialisation is idempotent*: Flyway applies only pending migrations, the playbook loads the catalogue dump only the first time, and Keycloak imports its realm only on its first start; the generated passwords are kept in a git-ignored folder, so that running the playbooks again reuses the values the databases were created with. (iii) *Results are stored in PostgreSQL*, not in memory, and pending jobs wait in Kafka, so users get their recommendations right after a restart. (iv) *The public address is stable*: the edge node keeps a reserved static IP, which Keycloak's tokens, the login redirects and the TLS certificate all name.

PLACEHOLDER. Still to build and confirm: the stop and start procedure ([TODO: Terraform `desired_status` or `gcloud`]), a static address for the control-plane node (its public IP is ephemeral, and the API server certificate and the kubeconfig contain it), refreshing the Ansible inventory after a restart, and a clean shutdown order. [TODO: details, recovery-time objective].

Validation. (1) Create users through the UI, set genres, watched movies and follows, and let the agent cache recommendations. Record row counts and checksums (`pg_dump | sha256sum`) for each database, plus the Keycloak user list. (2) Stop all VMs ([TODO: procedure]). (3) Start them, then wait until all nodes and pods are `Ready`. (4) Compare counts and checksums, log in as an existing user, and check that stored recommendations are served without inference. Results: [TODO: results, time to full recovery].

4 EVALUATION

Methodology. We measure throughput and latency as the number of concurrent clients increases. Load is generated with [TODO: tool, e.g. k6 [2] or Locust] from a separate VM in the same region as the cluster, so that the generator does not compete with the system for CPU and the wide-area network does not distort latencies. Each virtual user runs in a closed loop with [TODO: think time, e.g. 0 s / 1 s] and draws requests from a weighted mix: browsing `GET /movies` ([TODO: 40]%), search `GET /movies/search` ([TODO: 25]%), profile reads `GET /profiles/{id}` and `/watched` ([TODO: 20]%), profile writes ([TODO: 5]%), and recommendation requests ([TODO: 10]%). Recommendations are reported separately as *cached* (precomputed by the background agent) or *on demand* (with an inference call). Before the run, [TODO: N] test users are created in Keycloak. Each virtual user gets an access token through a dedicated load-test client with direct access grants ([TODO: to confirm: the realm's current clients have direct access grants disabled, so a dedicated client, or another way to obtain tokens, is needed]) and refreshes it before the 5-minute expiry, so token issuance stays out of the measured path. We step through 1, 10, 50, 100, 200 and [TODO: N] concurrent clients. Each step has a [TODO: 60] s warm-up and [TODO: 180] s of steady state and is repeated [TODO: 3] times. We report throughput (req/s) and p50/p95/p99 latency, overall and per endpoint. Prometheus collects CPU, memory, JVM, connection-pool and PostgreSQL metrics during each run to find the bottleneck.

Setup. The cluster of Section 2.3: one control-plane VM (`e2-standard-2`) and two workers (`e2-standard-4`, 4 vCPUs on 2 physical cores each), one replica per service, a 768 MiB memory limit per Quarkus service, and Ollama running Qwen2.5 3B on the CPUs, without a GPU, one request at a time. [TODO: database resources, any change made for the tests].

Results. Figure 2 and Table 4 summarise the results. One data point is already known: on the two `e2-standard-4` workers, a single on-demand recommendation took about 4.5 minutes, almost all of it the model reading its 2 000-token prompt on the CPU (27 s to read the request and about 4 minutes to choose among the candidates), which is why requests are asynchronous and why the background

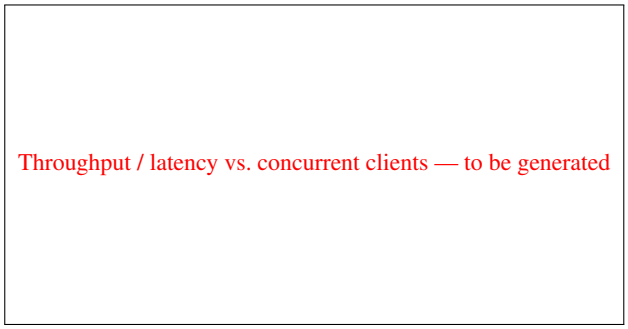


Figure 2: Throughput (req/s) and p95 latency as the number of concurrent clients grows.

Table 4: Throughput and latency for the full request mix.

Clients	req/s	p50 (ms)	p95 (ms)	p99 (ms)
1	[TODO:]	[TODO:]	[TODO:]	[TODO:]
10	[TODO:]	[TODO:]	[TODO:]	[TODO:]
50	[TODO:]	[TODO:]	[TODO:]	[TODO:]
100	[TODO:]	[TODO:]	[TODO:]	[TODO:]
200	[TODO:]	[TODO:]	[TODO:]	[TODO:]

agent precomputes the recommendations shown on login.

PLACEHOLDER. Discussion, once the data is available. Where throughput stops growing (saturation point) and latency rises sharply. Which component saturates first (APIs' CPU, PostgreSQL connection pool, search queries on the GIN/trigram indexes, or the inference engine), supported by the Prometheus metrics. How much slower on-demand recommendations are than cached ones, and so how much the background agent helps. Variance across repetitions. **[TODO:** numbers].

5 CONCLUSION

We presented a Movie Recommendation Hub built as four Quarkus microservices with a Next.js frontend, Keycloak authentication, one PostgreSQL database per service and Kafka between the recommendation service and its worker. It runs locally with one command in Docker, and in Google Cloud on a self-hosted Kubernetes cluster created with Terraform and Ansible and monitored with Prometheus and Grafana. Recommendations come from Qwen2.5 3B served by Ollama, on demand and precomputed by a background agent. Persistent volumes and idempotent initialisation are meant to let the whole infrastructure be stopped and restarted without losing state (Section 3; **[TODO:** validation result]). The evaluation shows **[TODO:** saturation point and main bottleneck]. As future work, we would consider GPU nodes for the inference engine, which would cut a recommendation from minutes to seconds, autoscaling of the stateless services, and a CI/CD pipeline.

DECLARATION OF USE OF AI

PLACEHOLDER. State which AI tools were used (e.g. for coding assistance, debugging, or revising the text of this report), for which tasks, and that all output was reviewed and validated by the group, who take full responsibility for the content.

REFERENCES

[1] Rounak Banik. The movies dataset. Kaggle, `https://www.kaggle.com/datasets/rounakbanik/the-movies-dataset`, 2017.

[2] Grafana Labs. Grafana k6 documentation. `https://grafana.com/docs/k6/latest/`. Accessed October 2026.

[3] Prometheus Authors. Prometheus: Monitoring system and time series database. `https://prometheus.io/docs/introduction/overview/`. Accessed October 2026.

[4] Chris Richardson. Pattern: Database per service. `https://microservices.io/patterns/data/database-per-service.html`. Accessed October 2026.

[5] Nat Sakimura, John Bradley, Michael B. Jones, Breno de Medeiros, and Chuck Mortimore. OpenID Connect Core 1.0. OpenID Foundation, `https://openid.net/specs/openid-connect-core-1_0.html`, 2014.

[6] The Kubernetes Authors. Persistent volumes. Kubernetes documentation, `https://kubernetes.io/docs/concepts/storage/persistent-volumes/`. Accessed October 2026.

[7] The Longhorn Authors. Longhorn: Cloud native distributed block storage for Kubernetes. `https://longhorn.io/docs/`. Accessed October 2026.